Assignment 2: Build an AI Agent for DevOps Repository Auditing

Course: MLOps & Production AI Engineering **Language:** Python **Submission:** Public GitHub repository

1. Objective

Develop an **AI-powered DevOps auditing agent** that autonomously inspects GitHub repositories, identifies DevOps best-practice violations, and generates an evidence-based report with actionable recommendations.

Agent cycle: Observe → Reason → Select Tool → Execute Tool → Interpret Results → Report.

Scope: Software engineering and DevOps only. ML training, datasets, model validation, experiment tracking, and other MLOps-specific tasks are excluded.

2. Functional Requirements

• Accept a public GitHub repository URL and inspect it automatically.

• Use an LLM to select inspection tools based on observations, including follow-up tool calls.

• Identify genuine violations, distinguish acceptable alternatives, and provide evidence and remediation.

• Produce the specified machine-readable JSON report without interactive prompts.

• Leave the inspected repository unchanged.

Agent requirement: A fixed checklist or static scanner followed by an LLM-written summary is insufficient. The LLM must help direct the investigation and tool selection.

3. Required Agent Architecture

Build an explicit Python agent/tool loop. Implement at least the following tools:

Tool	Purpose
list_files()	Discover repository files and structure
read_file(path)	Inspect relevant source or configuration
search_code(pattern)	Search for suspicious patterns
git_log()	Inspect commit history

Optional tools include run_pytest(), run_ruff(), git_status(), and GitHub Actions inspection. All tool outputs must be interpreted by the agent.

FRAMEWORK RESTRICTION: Prebuilt agent/orchestration platforms and frameworks (including n8n, LangChain, and LangGraph) are prohibited. Implement agent reasoning, tool dispatch, and control flow yourself in Python. Basic LLM provider SDKs are allowed.

4. DevOps Inspection Areas

Category	Example violations
Repository hygiene	Missing README, unsuitable .gitignore, tracked temporary/generated files
Dependency management	Inconsistent specifications, absent required lockfiles, weak version constraints
Code quality	Missing linting/formatting, obvious maintainability problems
Testing	Missing tests, failing tests, no CI test execution
GitHub Actions / CI	Incorrect triggers, ineffective gates, skipped validation
Security	Exposed credentials, unsafe workflow permissions
Configuration	Machine-specific paths, committed local settings, poor environment separation
Collaboration	Unsafe PR automation, lack of review guidance where relevant
Build and release	Missing build validation, risky release or deployment automation

Evidence rule: Do not treat an absent optional file/tool as proof of a defect when equivalent solutions exist. Do not infer remote branch protection settings from repository files alone.

5. Execution Interface and Output Contract

```
python agent.py --repo <github_repository_url> --output report.json
```

Run unattended. On a completed inspection, write valid JSON (including an empty violations array if none are detected). On fatal errors, exit nonzero with a useful error message.

```
{
  "repository": "https://github.com/example/demo-project",
  "violations": [
    {
      "rule_id": "SEC-001",
      "category": "security",
      "severity": "critical",
      "file": "config.py",
      "line": 17,
      "description": "Hardcoded API credential",
      "evidence": "Credential assigned directly in source code",
      "recommendation": "Use a secrets manager or environment variable"
    }
  ]
}
```

Each violation must contain all shown fields. Allowed severity values: critical, high, medium, low. Use JSON null for inapplicable file/line values; never reproduce secret values.

6. Hidden Repository Evaluation

The instructor will test the agent on **unseen GitHub repositories with known injected DevOps defects**: (A) obvious issues, (B) subtler CI/security/configuration defects, and (C) a mostly healthy repository to test false positives. Students will not be given hidden repositories or answer keys.

Precision = correctly identified violations / all reported violations. **Recall** = correctly identified violations / all known violations. Duplicate findings earn no extra credit; unsupported claims may be false positives. Tool-call traces may be inspected to verify genuine agent behavior.

7. Grading Rubric

Assessment component	Weight
Agent architecture and dynamic tool selection	20%
Inspection-tool implementation	15%
Hidden-repository detection (precision and recall)	30%
Evidence, severity, recommendations	15%
Reliability, error handling, safe execution	10%
JSON compliance, tests, documentation	10%
TOTAL	100%

The LLM provider or model choice does not affect the grade.

8. Submission Instructions

Submit only one public GitHub repository URL through the designated course submission channel. Include:

Required item	Description
agent.py	CLI entry point with required arguments
tools.py	Inspection tool implementations
Agent modules	LLM interaction, tool selection and execution loop
requirements.txt or pyproject.toml	Installable dependencies
.env.example	Required variable names, no real credentials
README.md	Exact installation, setup, architecture and execution instructions
tests/	Automated tests for tools and agent behavior
sample_report.json	Authentic example agent output

The evaluator must be able to clone your submission, install dependencies, configure supplied API credentials, and run the specified command on unseen repositories. No ZIP files, screenshots, or hand-written reports as substitutes for executable code.

9. Constraints and Safety

• Use Python and support public GitHub repositories.

• Do not modify targets or execute untrusted repository code without adequate isolation.

• Do not invent results when a check is unavailable.

• Never expose secrets in logs or output.

• Do not hardcode repository-specific findings or hidden answers.

• Handle missing files, API errors, invalid input, and timeouts gracefully.

10. Expected Learning Outcomes

Implement a controlled LLM–tool agent loop; design inspection tools; apply DevOps, GitHub, CI/CD, security and testing practices; support findings with evidence; evaluate detection precision/recall; and produce reliable structured output.

Success criterion: An agent that independently investigates unfamiliar repositories, identifies defensible DevOps violations, minimizes false positives, and produces actionable recommendations.